МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук
Кафедра автоматизированных систем управления

Лабораторная работа №6

по дисциплине «Машинное обучение»

Студент М-РИТ-25-1 _____ Станиславчук С. М.

(подпись, дата)

Руководитель

Доцент, д.т.н. _____ Сараев П. В.

(подпись, дата)

Липецк 2025

1. Задание кафедры

1) 1. Для множества данных из лр4 возьмите модель, которая использовалась в лр 5 (далее - базовая модель) и постройте ансамблевые модели, используя технологии стэкинга, бустинга, бэггинга.

2) Настройте гиперпараметры ансамблевых методов.

3) Постройте таблцы для сравнения значений адекватности моделей. Сравните адекватность моделей с базовой моделью и лучшей моделью из работы #5.

4) Сделайте вывод об ансамблевых методах.

2. Цель работы

Изучить подходы к построению моделей машинного обучнеия с использованием ансамблевых методов для повышения качества модели.

3. Ход работы

В качестве языка программирования для выполнения лабораторной работы, я выбрал Python.

Строим модель стекинг, используя функцию из пакета sklearn (StackingRegressor):

```
stacking_results = {}
from sklearn.ensemble import StackingRegressor

for target in ["Y1", "Y2"]:
    print(f"Building Stacking for {target} ...")
    estimators = [
        ("rf", RandomForestRegressor(n_estimators=200, max_depth=6,
random_state=42, n_jobs=-1)),
        ("svr", Pipeline([
            ("scaler", StandardScaler()),
            ("svr_est", SVR(kernel="rbf", C=1.0, gamma="scale"))
        ])),
        ("poly_ridge", Pipeline([
            ("poly", PolynomialFeatures(degree=poly_degree,
include_bias=False)),
            ("scaler", StandardScaler()),
            ("ridge", Ridge(alpha=1.0))
        ]))
    ]
    # final estimator
    final_est = LinearRegression()
    stack = StackingRegressor(estimators=estimators,
final_estimator=final_est, n_jobs=-1, passthrough=False)
    stack.fit(X_train, y_train[target])
    y_pred = stack.predict(X_test)
```

```
    stacking_results[target] = {"model": stack, "params":
"rf(200,d=6)+svr+poly_ridge", **evaluate(y_test[target], y_pred)}
```

Строим модель бустинг, используя функцию из пакета sklearn (HistGradientBoostingRegressor)

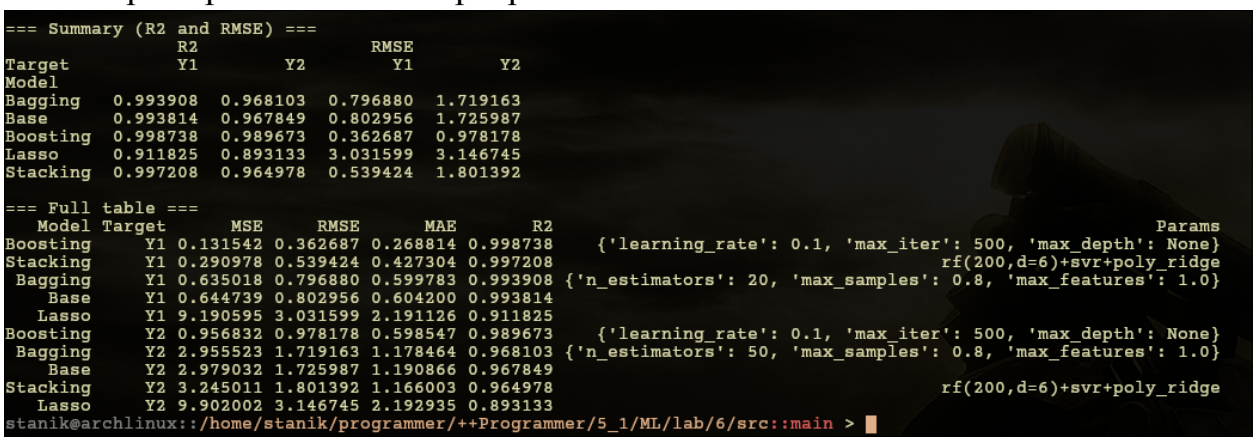
```
boosting_results = {}
for target in ["Y1", "Y2"]:
    print(f"Grid-search Boosting for {target} ...")
    best = None
    best_params = None
    best_score = -np.inf
    for lr in boost_params["learning_rate"]:
        for it in boost_params["max_iter"]:
            for md in boost_params["max_depth"]:
                hgb = HistGradientBoostingRegressor(learning_rate=lr,
max_iter=it, max_depth=md, random_state=42)
                hgb.fit(X_train, y_train[target])
                y_pred = hgb.predict(X_test)
                r2 = r2_score(y_test[target], y_pred)
                if r2 > best_score:
                    best_score = r2
                    best = hgb
                    best_params = {"learning_rate": lr, "max_iter": it,
"max_depth": md}
    y_pred = best.predict(X_test)
    boosting_results[target] = {"model": best, "params": best_params,
**evaluate(y_test[target], y_pred)}
```

Строим модель бэггинг, используя функцию из пакета sklearn (BaggingRegressor)

```
      def grid_search_bagging(X_tr, y_tr, X_val, y_val):
          best = None
          best_params = None
          best_score = -np.inf
          for n in bagging_grid["n_estimators"]:
              for ms in bagging_grid["max_samples"]:
                  for mf in bagging_grid["max_features"]:
                      bag = BaggingRegressor(
                          estimator=Pipeline([
                              ("poly", PolynomialFeatures(degree=poly_degree,
include_bias=False)),
                              ("scaler", StandardScaler()),
                              ("lr", LinearRegression())
                          ]),
                          n_estimators=n, max_samples=ms, max_features=mf,
random_state=42, n_jobs=-1
                      )
                      bag.fit(X_tr, y_tr)
                      y_pred = bag.predict(X_val)
                      r2 = r2_score(y_val, y_pred)
                      if r2 > best_score:
```

```
                            best_score = r2
                            best = bag
                            best_params = {"n_estimators": n, "max_samples": ms,
"max_features": mf}
          return best, best_params
```

Пример выполнения программы:



```
=== Summary (R2 and RMSE) ===
                R2                  RMSE
Target          Y1        Y2        Y1        Y2
Model
Bagging   0.993908  0.968103  0.796880  1.719163
Base      0.993814  0.967849  0.802956  1.725987
Boosting  0.998738  0.989673  0.362687  0.978178
Lasso     0.911825  0.893133  3.031599  3.146745
Stacking  0.997208  0.964978  0.539424  1.801392

=== Full table ===
   Model Target      MSE     RMSE      MAE       R2                                                                  Params
Boosting     Y1 0.131542 0.362687 0.268814 0.998738    {'learning_rate': 0.1, 'max_iter': 500, 'max_depth': None}
Stacking     Y1 0.290978 0.539424 0.427304 0.997208                                                rf(200,d=6)+svr+poly_ridge
 Bagging     Y1 0.635018 0.796880 0.599783 0.993908 {'n_estimators': 20, 'max_samples': 0.8, 'max_features': 1.0}
    Base     Y1 0.644739 0.802956 0.604200 0.993814
   Lasso     Y1 9.190595 3.031599 2.191126 0.911825
Boosting     Y2 0.956832 0.978178 0.598547 0.989673    {'learning_rate': 0.1, 'max_iter': 500, 'max_depth': None}
 Bagging     Y2 2.955523 1.719163 1.178464 0.968103 {'n_estimators': 50, 'max_samples': 0.8, 'max_features': 1.0}
    Base     Y2 2.979032 1.725987 1.190866 0.967849
Stacking     Y2 3.245011 1.801392 1.166003 0.964978                                                rf(200,d=6)+svr+poly_ridge
   Lasso     Y2 9.902002 3.146745 2.192935 0.893133
stanik@archlinux::/home/stanik/programmer/++Programmer/5_1/ML/lab/6/src::main >
```

3. Вывод

Линейная регрессия и Lasso ожидаемо продемонстрировали невысокую точность. Для Y1 R2 находится около 0.91, а RMSE — около 3.03. Для Y2 наблюдается аналогичная картина: R2 = 0.893, RMSE = 3.15. Это подтверждает, что базовые линейные модели недостаточно хорошо улавливают нелинейные зависимости. Полиномиальная модель GLM существенно улучшила качество, подняв R2 до 0.9938 и 0.9678.

Бэггинг показал стабильное повышение точности по сравнению с базовыми моделями R2 = 0.9939, 0.9681. По уровню качества Бэггинг практически сравнялся с GLM. Уменьшение разброса предсказаний за счёт усреднения ансамбля деревьев обеспечивает значительное снижение ошибки.

Бустинг стал абсолютным лидером среди всех методов. R2 = 0.9987, 0.9897. Модель обеспечивает почти идеальное приближение, что подтверждает способность бустинга последовательно исправлять ошибочные области предыдущих слабых моделей.

Стекинг показал результаты лучше GLM, но хуже Бэггинга и Бустинга R2 = 0.9972, 0.9649. Лучше отработал на Y1, потому что нелинейная структура выражена сильнее.

Приложение 1

```
# Изучить подходы к построению моделей машинного обучнеия с применением методов
# регуляризации для борьбы с переобучением.

# https://www.geeksforgeeks.org/machine-learning/lasso-vs-ridge-vs-elastic-net-ml/

# 1. Для множества даных из л.р. 4 для одной из моделей (далее - базовая модель)
# постройте модели с регуляризацией (L1-регуляризация (LASSO); L2-регуляризация
# (Ridge); комбинированная регуляризация (ElastiocNet)).

# **Старье**

# 1.1 Загрузите из UCI ML Repository базу данных Energy efficiency.
# http://archive.ics.uci.edu/ml/datasets/Energy+efficiency
# В этой базе представлены результаты исследования энергоэффективности
# зданий в виде отопительной (heating) и холодильной (cooling) нагрузки
# в зависимости от параметров

import pandas as pd
import numpy as np

df = pd.read_csv("energy_efficiency.csv")

X = df.iloc[:, :-2]  # first 8 columns (features)
y = df.iloc[:, -2:]  # last 2 columns (targets)

print("Dataset loaded locally!")
print(X.head())
print(y.head())


from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)


# **Конец старья**

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Lasso, Ridge, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression, Lasso, Ridge, ElasticNet
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

models = {
    "LinearRegression": LinearRegression(),
    "Lasso": Lasso(random_state=42),
    "Ridge": Ridge(random_state=42),
    "ElasticNet": ElasticNet(random_state=42)
}

for target in ["Y1", "Y2"]:
    print(f"\n=== {target} ===")
    for name, model in models.items():
        model.fit(X_train, y_train[target])
        y_pred = model.predict(X_test)
        mse = mean_squared_error(y_test[target], y_pred)
        r2 = r2_score(y_test[target], y_pred)
        print(f"{name:17s} | MSE: {mse:10.3f} | R²: {r2:.3f}")
```

```
# 2. Настройте гиперпараметры регуляризации.
print("\n")
step = 10
models = {

    "Base": (LinearRegression(), {}),

    "Ridge": (Ridge(), {"model__alpha": np.logspace(-3, 3, step)}),

    "Lasso": (Lasso(max_iter=10000), {"model__alpha": np.logspace(-3, 3, step)}),

    "ElasticNet": (
        ElasticNet(max_iter=10000),
        {
            "model__alpha": np.logspace(-3, 3, step),
            "model__l1_ratio": np.linspace(0.1, 0.9, 9)
        },
    ),
}


# 3. Постройте таблицы для сравнения значений адекватности моделей. Сравните
# адекватеонсть моделй с базоваой моделью
results = []

target_columns = ['Y1', 'Y2']

for target in target_columns:
    for name, (model, params) in models.items():
        pipe = Pipeline([
            ("scaler", StandardScaler()),
            ("model", model)
        ])

        grid = GridSearchCV(pipe, params, cv=5, scoring="r2", n_jobs=-1)
        grid.fit(X_train, y_train[target])

        y_pred = grid.predict(X_test)

        mse = mean_squared_error(y_test[target], y_pred)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(y_test[target], y_pred)
        r2 = r2_score(y_test[target], y_pred)

        results.append({
            "Target": target,
            "Model": name,
            "Best Params": grid.best_params_,
            "MSE": mse,
            "RMSE": rmse,
            "MAE": mae,
            "R²": r2
        })

df_results = pd.DataFrame(results)
print(df_results.to_string(index=False))
```